Infraestructura TI

Solemne 1

Sistema IoT para el registro y monitoreo de estacionamientos

Integrantes:

Luis Bahamonde Villalobos – Rut: 21.858.081-5

Maira Brante Collao – Rut: 22.098.287-4

Jorge Castillo Miranda – Rut: 21.301.367

Anaís Tuteleers Calderón – Rut: 21.304.704-3

Christopher Torrejón Allende – Rut: 21.774.845-3

Asignatura: Infraestructura TI

NRC: 8571

Profesor: Luis Delgado Ayala

Fecha de entrega: 28 de octubre de 2026

Universidad Andrés Bello

Facultad de Ingeniería

Tabla de contenido

1. Introducción ... 3
2. Objetivos ... 3
2.1. Objetivo general ... 3
2.2. Objetivos específicos ... 3
3. Alcance y descripción del sistema ... 3
4. Hardware y adquisición de eventos ... 4
4.1. Componentes ... 4
4.2. Cableado ... 4
4.3. Preparación y diagnóstico ... 5
4.4. Registro y publicación ... 5
5. Arquitectura de microservicios ... 5
6. API, almacenamiento y plataforma web ... 6
6.1. Persistencia local y sincronización ... 6
6.2. Endpoints identificados ... 6
6.3. Modelo de datos y dashboard ... 7
7. Seguridad y despliegue ... 7
8. Pruebas y resultados ... 7
8.1. Detección de movimiento ... 7
8.2. Estacionamientos simulados ... 8
8.3. Historial y otras funciones de la plataforma ... 9
8.4. Ejecución de microservicios ... 10
8.5. Estado de sincronización ... 10
9. Evaluación de consumo, capacidad y costos ... 11
9.1. Escenario de uso ... 11
9.2. Estimación de consumo diario y mensual ... 11
9.3. Límites de las capas gratuitas ... 12
9.4. Mediciones y limitaciones ... 13
9.5. Posibles optimizaciones ... 14
9.6. Proyección de crecimiento y costos ... 15
10. Conclusiones ... 17
11. Referencias ... 18
12. Anexo ... 18

1. Introducción

El presente proyecto consiste en un sistema IoT para registrar y visualizar información relacionada con un estacionamiento. Utiliza una Raspberry Pi 4 y un sensor PIR HC-SR501 para detectar movimiento. También incluye programas en Python, servicios en Docker, comunicación mediante MQTT, almacenamiento local en SQLite, una base de datos Firestore y una página web publicada con Vercel.

El sistema trabaja con dos tipos de información: las detecciones reales de un sensor PIR y los datos simulados de ocho espacios de estacionamiento. Estos ocho espacios no corresponden a ocho sensores físicos. Sus estados de ocupación y distancias se generan mediante un simulador.

2. Objetivos

2.1. Objetivo general

Desarrollar y documentar un sistema IoT que registre eventos de movimiento y permita consultar información de estacionamientos mediante sensores, programas de procesamiento y una plataforma web.

2.2. Objetivos específicos

- Integrar un sensor PIR con una Raspberry Pi 4 y registrar eventos con fecha, hora y duración de activación.
- Enviar eventos mediante MQTT y distribuir las tareas entre diferentes servicios.
- Guardar los registros en SQLite y enviar los datos correspondientes a Cloud Firestore.
- Mostrar por separado eventos físicos e información simulada en un dashboard web.
- Revisar el consumo de recursos del sistema y sus posibles límites, riesgos y mejoras.

3. Alcance y descripción del sistema

El sensor PIR detecta movimiento y la Raspberry Pi registra los eventos en un archivo CSV. Además, envía mensajes mediante MQTT para que los demás servicios puedan recibirlos y procesarlos. Por otro lado, un simulador genera datos para ochos espacios virtuales (P01-P08). La información se almacena y se presenta en la plataforma web.

La página web muestra por separado los datos del sensor físico y los espacios simulados. Aunque aparezcan referencias a vehículos en la interfaz, el sensor PIR detecta movimiento y no puede identificar por sí solo qué lo provocó.

4. Hardware y adquisición de eventos

4.1. Componentes

Para el montaje físico se utilizaron una Raspberry Pi 4, un sensor PIR HC-SR501, una mini protoboard, cables de conexión, una fuente de alimentación de 5 V y una tarjeta microSD de 16GB. La Raspberry también cuenta con ventilación.

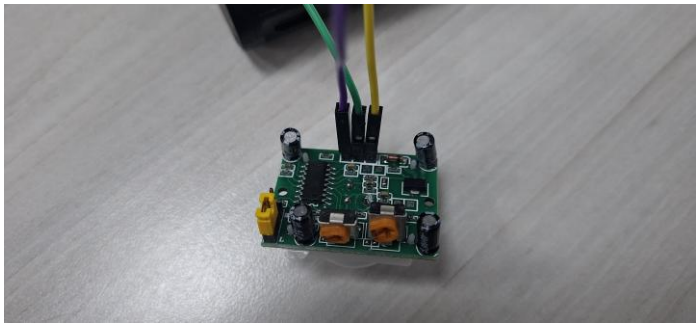


Figura 1. Montaje del sensor PIR utilizado en el proyecto.

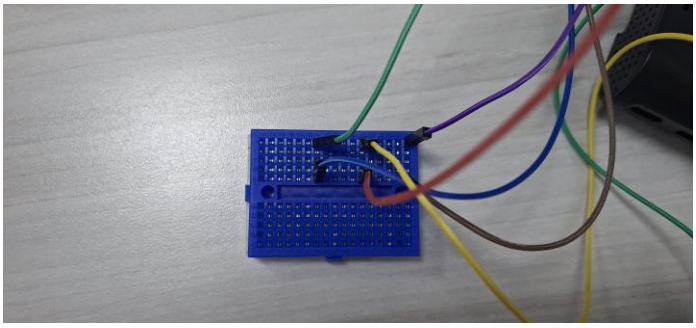


Figura 2. Vista del módulo PIR HC-SR501.

4.2. Cableado

Terminal PIR	Conexión prevista	Propósito
VCC	Pin físico 2 (5 V)	Alimentación
OUT	Pin físico 11 (GPIO17)	Entrada digital
GND	Pin físico (GND)	Referencia de tierra

Tabla 1. Cableado del sistema y propósito.

El sensor PIR se conecta a la Raspberry Pi mediante tres terminales: VCC, OUT y GND. El terminal VCC recibe la alimentación de 5 V, mientras que GND corresponde a la conexión a tierra. Por otro lado, OUT envía la señal de detección al GPIO17 (pin físico 11), que permite a la Raspberry identificar los cambios de estado del sensor.

4.3. Preparación y diagnóstico

La tarjeta microSD se preparó con Raspberry Pi Imager. Después se accedió a la Raspberry mediante SSH, se actualizó el sistema, se verificó el modelo de la placa y se instaló Docker. Para comprobar el sensor se utilizó pir_diag.py, que lee el valor digital de GPIO17 cada medio segundo. Los valores 0 y 1 permiten observar los cambios de estado del sensor.

4.4. Registro y publicación

El programa estacionamientoF_mqtt.py lee la señal del sensor mediante GPIO17. Tras un periodo inicial de calibración, registra los eventos en un archivo CSV y los publica mediante MQTT. De esta manera, las detecciones pueden llegar a los servicios encargados de procesar y almacenar la información.

La duración registrada indica cuánto tiempo estuvo activa la señal del PIR. Esto no equivale necesariamente al tiempo que un vehículo permanece estacionado.

5. Arquitectura de microservicios

El archivo docker-compose.yml define cuatro servicios: mosquitto, simulator, processor y api. Mosquitto, processor y api tienen configurado el reinicio automático con “unless-stopped”, mientras que el simulador no. La configuración incluye una comprobación de estado (healthcheck) para Mosquitto, pero no define una equivalente para los otros dos servicios principales.

Componente	Responsabilidad
Raspberry/PIR	Lee GPIO17, registra eventos y publica MQTT desde el host
simulator	Produce datos simulados de ocho plazas para la demostración
mosquitto	Broker que distribuye mensajes MQTT
processor	Interpreta eventos y produce estados y resúmenes
api	Recibe mensajes, registra eventos en SQLite, expone REST y sincroniza pendientes
cloud-api	Valida token y metadatos de dispositivo; escribe eventos en Firestore
web	Interfaz Next.js para consulta, visualización y control de acceso

Tabla 2. Componentes de la arquitectura y responsabilidad.

En términos generales, la información sigue este recorrido: el sensor PIR y el simulador envían mensajes a Mosquitto; luego, los servicios los reciben y procesan. La API local guarda datos en SQLite y puede sincronizarlos con la API de la nube. Finalmente, Firestore almacena los registros que consulta la plataforma web. Algunos servicios trabajan al mismo tiempo, por lo que no todos los mensajes siguen una única secuencia.

Los servicios cumplen funciones distintas: el simulador genera datos de las ocho plazas, Mosquitto distribuye mensajes MQTT, processor calcula estados y resúmenes, y la API local guarda eventos y gestiona su envío a la nube. Por lo tanto, la cantidad de microservicios no depende de la cantidad de sensores físicos.

El programa del PIR se ejecuta en la Raspberry y publica eventos en el tópico parking/pir/pir-entrada-01/event. Los espacios simulados publican sus distancias en parking/sensors/{plaza}/distance; el procesador envía los estados a parking/state/{plaza} y el resumen a parking/summary. Estos tópicos organizan el intercambio de información.

6. API, almacenamiento y plataforma web

6.1. Persistencia local y sincronización

La API local está desarrollada con FastAPI y utiliza SQLite para guardar los eventos. Cada registro cuenta con un identificador que ayuda a evitar duplicados cuando se envía nuevamente. Si se configuran la dirección de la API en la nube y el token de acceso, el sistema envía mediante HTTPS los eventos pendientes. La API en la nube revisa el token y los datos recibidos antes de almacenarlos en Firestore.

6.2. Endpoints identificados

Método / ruta	Función
GET /health	Estado básico de API local y Cloud API
GET /parking/spaces	Consulta de espacios en API local
GET /parking/spaces/{space_id}	Detalle de una plaza simulada
GET /parking/summary	Resumen de ocupación
GET /parking/events	Eventos locales con parámetro limit
GET /parking/sync-status	Consulta de estado de sincronización
POST /ingest (Cloud API)	Ingreso autenticado y validado a Firestore

Tabla 3. Endpoints del proyecto y utilidad.

6.3. Modelo de datos y dashboard

Los eventos que llegan desde el PIR se identifican por dispositivo, fecha, tipo de evento y un identificador único. En los registros físicos también aparece el campo simulated con valor false. En cambio, el simulador genera los estados de los ochos espacios virtuales, que se muestran por separado en el panel web.

El dashboard está desarrollado por Next.js y se publica en Vercel. Muestra un resumen general, un historial con filtros por fecha, el estado de los dispositivos y alertas de actividad. El acceso utiliza Firebase Authentication. La sección de administración muestra la organización, sede y rol de la cuenta, pero no permite todavía gestionar usuarios o dispositivos desde esa pantalla.

La API local registra los eventos en SQLite antes de enviarlos a la nube. Para evitar registros duplicados, utiliza identificadores que permiten reconocer si un mensaje ya fue almacenado. Además, cuando se pierde la conexión, los eventos pendientes se conservan localmente para poder enviarlos cuando se restablezca la comunicación.

7. Seguridad y despliegue

El sistema utiliza Firebase Authentication para controlar el acceso a la plataforma web y distingue entre los roles de administrador y visualizador. Además, la API verifica el token del dispositivo antes de guardar los eventos en Firestore. Las credenciales se configuran mediante variables de entorno, lo que permite mantenerlas separadas del código fuente.

Para preparar la Raspberry Pi se utiliza Raspberry Pi OS y se configura el acceso remoto mediante SSH. Por otro lado, Docker Compose permite ejecutar los distintos servicios del sistema. El proyecto también incluye instrucciones de instalación y configuración para facilitar su puesta en funcionamiento.

8. Pruebas y resultados

Para revisar el funcionamiento del sistema, se ingresó a la plataforma web Parking Smart el 28 de octubre de 2026. Durante esta revisión se comprobaron las distintas secciones del dashboard y los datos que se encontraban disponibles.

8.1. Detección de movimiento

En el panel principal se observaron los eventos registrados por el sensor PIR conectado a la Raspberry Pi 4. La página mostraba una muestra de 96 eventos de movimiento, junto con la fecha y hora de la última detección, correspondiente a las 9:52:57 a. m.

También se visualizaron los eventos más recientes del sensor, lo que permitió comprobar que los registros podían consultarse desde la plataforma web.

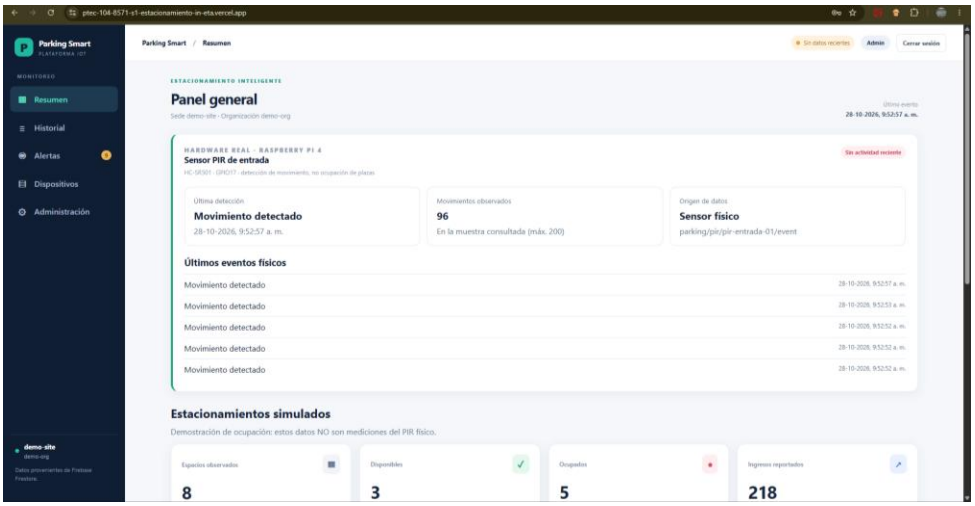


Figura 3. Panel principal y eventos del sensor PIR.

8.2. Estacionamientos simulados

La plataforma también muestra ocho espacios de estacionamiento simulados, identificados desde P01 hasta P08.

Al momento de la revisión, tres espacios aparecían disponibles y cinco ocupados, lo que corresponde al 62,5 % de ocupación. Además, se mostraban 218 ingresos reportados en el último resumen recibido mediante MQTT.

Estos datos corresponden a la simulación y se presentan por separado de las detecciones del sensor físico.

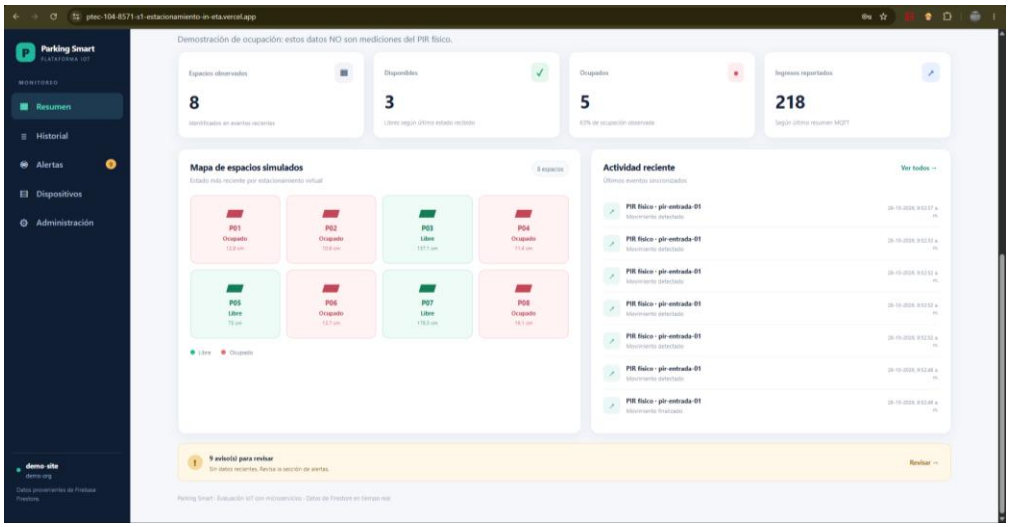


Figura 4. Estado y disponibilidad de los estacionamientos simulados.

8.3. Historial y otras funciones de la plataforma

En la sección de historial se observaron 228 registros en la muestra consultada. La tabla permite revisar información como la fecha, el tipo de evento y los datos asociados a cada registro.

Por otro lado, el apartado de alertas mostraba nueve avisos: uno por falta de datos recientes y ocho por lecturas antiguas de los estacionamientos simulados.

Finalmente, se revisaron las secciones de dispositivos y administración. Estas permiten consultar información de los sensores, los espacios simulados y el usuario que ha iniciado sesión.

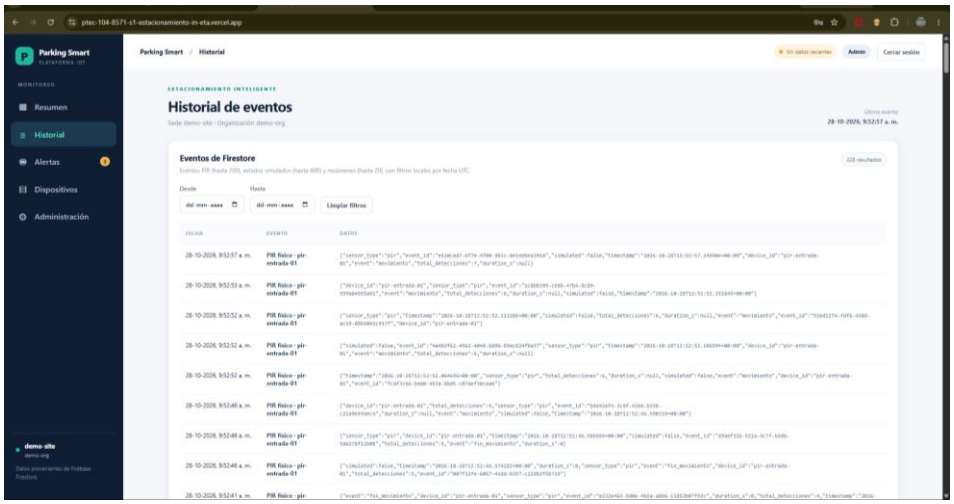


Figura 5. Historial de eventos.

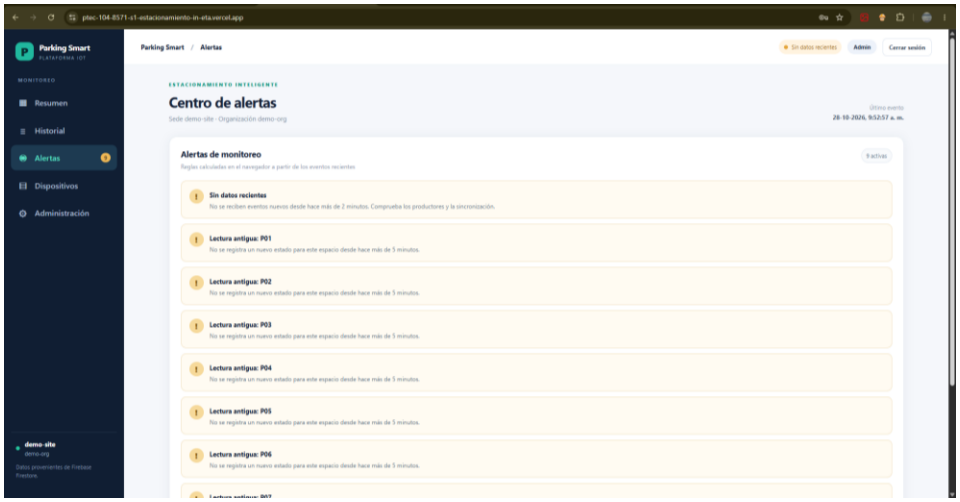


Figura 6. Alertas de monitoreo.

8.4. Ejecución de microservicios

Para revisar el estado de los contenedores se ejecutó docker compose ps en la Raspberry Pi. En la Figura 7 se observan tres servicios activos: parking-api, parking-mosquitto y parking-processor. Mosquitto aparece además con estado Healthy, lo que indica que superó su comprobación de estado.

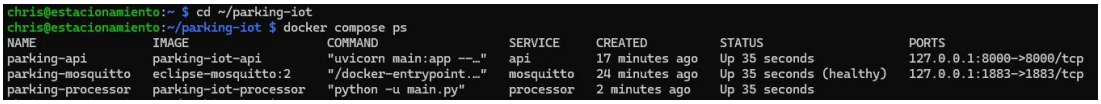


Figura 7. Estado de Docker Compose antes de iniciar el simulador.

Luego se ejecutó docker compose --profile manual up -d simulator. En la Figura 8 se observan los cuatro contenedores activos, incluido parking-simulator. Esto confirma que el simulador se puede iniciar por separado de los otros servicios.

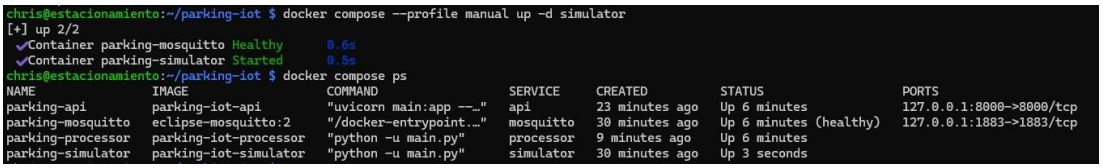


Figura 8. Estado de Docker Compose con el simulador en ejecución.

Los resultados permiten comprobar que los servicios se encontraban en ejecución al momento de la prueba. Además, se verificó que el simulador puede iniciarse por separado, sin afectar la ejecución de los demás contenedores.

8.5. Estado de sincronización

Se consultó el endpoint /parking/sync-status desde la Raspberry Pi. Al momento de la consulta, la API local mostraba 23.728 eventos contabilizados, cero eventos pendientes de envío y la conexión con la nube configurada (cloud_configured: true). Estos valores corresponden al estado observado en esa consulta y no son una medición directa de la cantidad de documentos almacenados en Firestore.

El proyecto utiliza almacenamiento local en SQLite para conservar registros mientras no haya conexión. La información proporcionada durante las pruebas indica que los datos pendientes se sincronizan posteriormente al recuperarse la conectividad. La consulta con cero pendientes es compatible con ese funcionamiento, aunque por sí sola no sustituye una prueba controlada de desconexión y recuperación.

9. Evaluación de consumo, capacidad y costos

Para evaluar el consumo y la capacidad de Parking Smart, se realizarán estimaciones sobre la cantidad de datos generados, las operaciones de almacenamiento y las solicitudes a los servicios en la nube. Estos resultados se compararán con los límites gratuitos de Firebase y Vercel. Además, se analizarán posibles mejoras para reducir el consumo y los costos que tendría ampliar el sistema.

9.1. Escenario de uso

Para evaluar el consumo de recursos de Parking Smart, se consideran dos escenarios de funcionamiento. El primero corresponde a una demostración en la que el simulador permanece activo durante dos horas al día. El segundo supone que funciona de manera continua, durante las 24 horas.

En ambos casos se considera una Raspberry Pi 4 con un sensor PIR físico y ocho espacios de estacionamiento simulados. Además, para realizar los cálculos se utilizan las frecuencias de generación de eventos y las condiciones de uso de la plataforma.

Estos escenarios permiten estimar la cantidad de información que podría almacenarse en Firestore y las solicitudes que recibiría Vercel. Los resultados corresponden a estimaciones y no a mediciones reales del consumo.

9.2. Estimación de consumo diario y mensual

Para estimar el consumo de Parking Smart se considera la cantidad de eventos que genera el sistema durante su funcionamiento. Estos incluyen los cambios de estado de los estacionamientos simulados, los resúmenes de ocupación e ingresos y las detecciones del sensor PIR físico.

En el escenario de funcionamiento continuo, se estima un total aproximado de 49.032 eventos diarios. De ellos, 22.738 corresponden a estados de los espacios simulados, 26.194 a resúmenes y 100 a eventos del PIR, considerados como un supuesto para realizar el cálculo.

Por otro lado, cuando el simulador funciona durante dos horas al día, se estima un total de 4.178 eventos diarios. En ambos casos se considera un mes de 30 días para calcular el consumo mensual.

Como el sistema está diseñado para enviar cada evento a la nube mediante una solicitud independiente, la cantidad de eventos también permite estimar las escrituras en Firestore y las invocaciones de la API en Vercel.

Concepto	Simulador 2h / día	Simulador 24h / día
Eventos estimados por día	4.178	49.032
Eventos estimados por mes	125.340	1.470.960
Escrituras potenciales en Firestore por día	4.178	49.032
Invocaciones potenciales de ingesta en Vercel por día	4.178	49.032

Tabla 4. Consumo diario y estimación por uso 24h / día.

Además de las escrituras e invocaciones, se estimó el consumo de lecturas y almacenamiento de Firestore. Para el escenario de funcionamiento continuo, se consideraron dos usuarios consultando la plataforma durante ocho horas diarias, obteniendo una estimación de 67.096 lecturas por día.

Por otro lado, el almacenamiento generado se estimó en aproximadamente 234 MB diarios, considerando tanto los datos guardados como el espacio adicional utilizado por los índices de Firestore.

Estas cifras representan el consumo esperado bajo las condiciones definidas en el escenario de uso. El consumo real puede variar según el tiempo que permanezca activo el simulador, la cantidad de movimientos detectados por el PIR, las consultas realizadas por los usuarios y la disponibilidad de conexión a la nube.

9.3. Límites de las capas gratuitas

Para evaluar la capacidad del sistema, se compararon las estimaciones de consumo con los límites gratuitos de Firebase y Vercel.

Para Cloud Firestore en el plan gratuito Spark se consideran 50.000 lecturas, 20.000 escrituras y 20.000 eliminaciones diarias, además de 1 GiB de almacenamiento. En el caso de Vercel, se utiliza como referencia el límite de invocaciones supuesto en el modelo de estimación; su facturación y uso efectivo deben comprobarse en el panel del proyecto.

Recurso	Límite gratuito	Simulador 2h / día	Simulador 24h / día
Escrituras Firestore	20.000/día	4.178/día	49.032/día
Invocaciones Vercel	1.000.000/mes	125.340/mes	1.470.960/mes

Tabla 5. Límite gratuito por servicio vs uso del proyecto.

En el escenario de dos horas diarias de simulación, las 4.178 escrituras estimadas se mantienen dentro del límite gratuito de Firestore. Lo mismo ocurre con las 125.340 invocaciones mensuales estimadas para Vercel.

Sin embargo, en el escenario de funcionamiento continuo se generarían aproximadamente 49.032 escrituras diarias y 1.470.960 invocaciones mensuales, superando los límites gratuitos de ambos servicios.

Además, para el escenario de funcionamiento continuo se estimaron aproximadamente 67.096 lecturas diarias y 234 MB de almacenamiento nuevo por día. Bajo estas condiciones, las lecturas también superarían el límite gratuito de Firestore. Por otro lado, si los registros se conservan de manera indefinida, el almacenamiento disponible podría agotarse en pocos días.

Estos resultados muestran que el escenario de dos horas diarias presenta un consumo menor y se mantiene dentro de los límites de escrituras e invocaciones. Sin embargo, para mantener el sistema funcionando de manera continua, sería necesario aplicar mejoras que permitan reducir el consumo de recursos.

9.4. Mediciones y limitaciones

La evaluación contemplaba comparar las estimaciones con mediciones reales durante siete días. Sin embargo, el retraso en la disponibilidad de materiales impidió completar ese período de seguimiento.

Como evidencia puntual se revisaron las métricas de Cloud Firestore el 28 de octubre de 2026. En el panel de operaciones se mostraban aproximadamente 79 mil lecturas, 17 mil escrituras y una eliminación dentro del intervalo representado. También se registraron unas 3,1 mil autorizaciones y 36 rechazos de reglas; el máximo observado fue de cuatro conexiones activas y 28 escuchas de instantáneas.

En otra captura, el panel de cuotas del proyecto Spark mostraba aproximadamente 43 mil lecturas, equivalentes al 86,8 % de la cuota diaria de 50 mil, y 3,3 mil escrituras, equivalentes al 16,3 % del límite diario de 20 mil. La pantalla también indicaba que el máximo de lecturas observado en los últimos siete días había llegado al 100 % de la cuota al menos una vez. Estas cifras corresponden a pantallas y períodos distintos, por lo que no deben sumarse ni compararse directamente como si midieran lo mismo.

Los datos disponibles permiten identificar un consumo real considerable de lecturas, pero no reemplazan siete días de mediciones continuas. Tampoco se obtuvo una cifra verificada de almacenamiento o de invocaciones de Vercel para el mismo período. Por esta razón, las proyecciones de las secciones anteriores se mantienen como estimaciones y no se consideran valores medidos.

9.5. Posibles optimizaciones

A partir de las estimaciones realizadas, se identificaron algunas mejoras que podrían ayudar a reducir el consumo de recursos de Parking Smart, especialmente durante períodos de funcionamiento continuo.

Una de las principales propuestas consiste en agrupar los registros antes de enviarlos a la nube. En el modelo actual se considera el almacenamiento individual de cada evento. Como alternativa, se podría generar un resumen por minuto con la información de los estacionamientos y la cantidad de movimientos registrados, disminuyendo las escrituras en Firestore. Esta modificación exigiría revisar cómo se conserva el historial detallado.

Otra mejora sería enviar los registros por lotes, es decir, agrupar varios datos en una misma solicitud a la API de Vercel. Esto permitiría disminuir la cantidad de invocaciones necesarias para sincronizar la información.

Además, se podría revisar la configuración de los índices de Firestore, manteniendo solamente aquellos que sean necesarios para las consultas de la plataforma. Esto ayudaría a reducir el espacio de almacenamiento utilizado.

Una propuesta complementaria es publicar los estados simulados solo cuando cambien, en vez de repetir información sin modificaciones. Esto reduciría mensajes MQTT y procesamiento innecesario. No se asigna un porcentaje de ahorro porque no se realizaron mediciones específicas de esta alternativa.

Estas optimizaciones corresponden a propuestas de mejora y todavía no han sido implementadas. Por lo tanto, sus beneficios deberán comprobarse mediante nuevas pruebas y mediciones de consumo.

Recurso	Diseño actual (estimado)	Diseño propuesto (estimado)	Reducción
Escrituras Firestore por día	49.032	1.440	97,1%
Invocaciones Vercel por día	49.032	1.440	97,1%
Invocaciones Vercel por mes	1.470.960	43.200	97,1%

Tabla 6. Comparación del diseño actual vs diseño optimizado.

Para comparar el posible efecto de estas mejoras, se realizó una estimación considerando el escenario de funcionamiento continuo. Si el sistema generara y enviara un resumen por minuto, las escrituras diarias en Firestore podrían disminuir de 49.032 a 1.440. De manera similar, las invocaciones de la API en Vercel pasarían de 49.032 a 1.440 por día, lo que representa una reducción aproximada del 97,1 %.

Estos resultados son teóricos, ya que las optimizaciones todavía no se han aplicado. Además, sería necesario comprobar que los resúmenes permitan conservar la información requerida para consultar el historial de eventos.

9.6. Proyección de crecimiento y costos

Para evaluar cómo podría crecer Parking Smart, se proyectó el consumo con 1, 10, 100 y 1.000 módulos IoT equivalentes al prototipo. Cada módulo representa un sensor PIR físico y ocho plazas simuladas; no se supone que una sola plaza corresponda a un sensor físico.

La proyección supone que cada módulo mantiene el mismo patrón de generación de eventos durante 24 horas diarias. Se compara el modelo de consumo actual con una propuesta teórica de un resumen por minuto y por módulo.

Dispositivos	Diseño actual: escrituras/día	Diseño propuesto: escrituras/día
1	49.032	1.440
10	490.320	14.400
100	4.903.200	144.000
1.000	49.032.000	1.440.000

Tabla 7. Proyección teórica de escrituras diarias según la cantidad de dispositivos.

Los resultados muestran que la agrupación de registros podría reducir considerablemente las escrituras. No obstante, incluso el diseño propuesto superaría la cuota gratuita diaria de Firestore al crecer a 100 o 1.000 dispositivos.

Para evaluar alternativas de despliegue se comparan PaaS (Vercel Pro y Firestore) e IaaS (máquinas virtuales de Amazon EC2). PaaS reduce la necesidad de administrar servidores; en IaaS, el equipo debe gestionar configuración, actualizaciones, seguridad y respaldo de los sistemas.

Escenario	PaaS: referencia mensual	IaaS: referencia mensual
1 dispositivo	USD 20 + consumo adicional de Firestore	USD 21,31 aprox.
100 dispositivos	USD 20 + consumo adicional de Vercel/Firestore	USD 72,68 aprox.

Tabla 8. Comparación preliminar de infraestructura mensual, en USD y sin impuestos.

Los valores de PaaS incluyen únicamente el precio base de Vercel Pro (USD 20 mensuales, con crédito de uso) y dejan sin cuantificar los cobros variables de Firestore y los posibles cargos adicionales de Vercel. Los valores de IaaS son configuraciones hipotéticas: para un dispositivo se considera una EC2 t3.small con 30 GB EBS e IPv4 pública; para 100 dispositivos, dos t3.medium con 100 GB EBS e IPv4 pública. Ninguna de estas configuraciones fue sometida a pruebas de carga y no se incluyen transferencia, respaldos, administración ni posibles cargos extra.

Por lo tanto, estos subtotales no permiten determinar una alternativa definitivamente más económica. Los costos finales dependerían del tráfico real, las lecturas y escrituras, el almacenamiento y la capacidad de procesamiento requerida. Las optimizaciones descritas son propuestas aún no implementadas.

Al aumentar la cantidad de dispositivos, también podrían crecer las necesidades de cómputo, transferencia, almacenamiento y administración. La comparación sirve como presupuesto preliminar de infraestructura, no como un estado de resultados comerciales: no se dispone de ingresos proyectados ni de un dimensionamiento validado para calcular una rentabilidad del proyecto.

10. Conclusiones

Parking Smart permitió integrar una Raspberry Pi 4 con un sensor PIR físico y una plataforma para visualizar eventos de movimiento. El sistema también incorpora datos simulados de ocho espacios de estacionamiento, diferenciados de las detecciones reales. Esta separación es importante porque el sensor PIR identifica movimiento, pero no cuenta vehículos ni determina por sí mismo la ocupación de una plaza.

Las pruebas realizadas permitieron observar registros en el dashboard, ejecutar los microservicios con Docker Compose y consultar el estado de sincronización de la API local. En esta última consulta se contabilizaron 23.728 eventos y no se informaron registros pendientes. Asimismo, las métricas de Firestore mostraron un uso relevante de lecturas y escrituras, incluyendo un consumo próximo al límite diario de lecturas en el panel de cuotas.

El proyecto quedó limitado por la disponibilidad de un solo tipo de sensor físico, la ausencia de siete días completos de mediciones y la falta de pruebas controladas de las optimizaciones propuestas. Como trabajo futuro, se plantea incorporar un segundo tipo de sensor, medir el comportamiento de los servicios durante un período más largo y evaluar la agrupación de registros para reducir el consumo de la nube.

11. Referencias

[1] Google Firebase. «Uso y límites de Cloud Firestore». https://firebase.google.com/docs/firestore/quotas?hl=es (consulta: 28-10-2026).

[2] Google Cloud. «Precios de Cloud Firestore». https://cloud.google.com/firestore/pricing (consulta: 28-10-2026).

[3] Vercel. «Pro Plan». https://vercel.com/docs/plans/pro-plan (consulta: 28-10-2026).

[4] Amazon Web Services. «Amazon EC2 Pricing». https://aws.amazon.com/ec2/pricing/ (consulta: 28-10-2026).

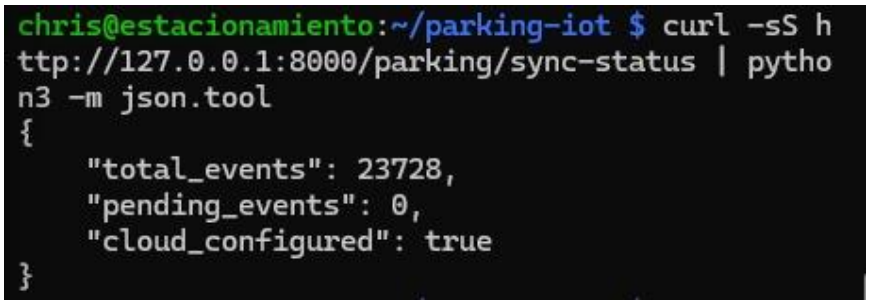
[5] Amazon Web Services. «Amazon EBS Pricing». https://aws.amazon.com/ebs/pricing/ (consulta: 28-10-2026).

[6] Raspberry Pi. «Raspberry Pi Documentation». https://www.raspberrypi.com/documentation/ (consulta: 28-10-2026).

[7] Docker. «Docker Compose». https://docs.docker.com/compose/ (consulta: 28-10-2026).

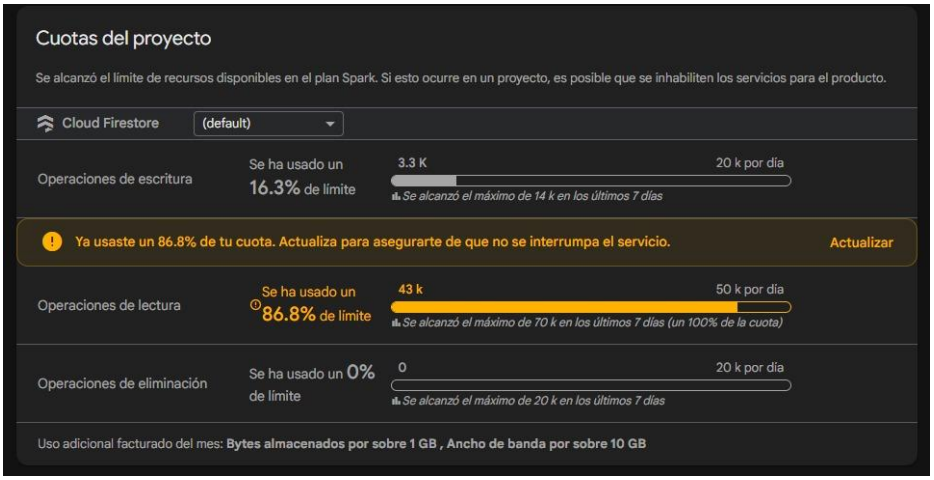
12. Anexo

Anexo A. Consulta de sincronización



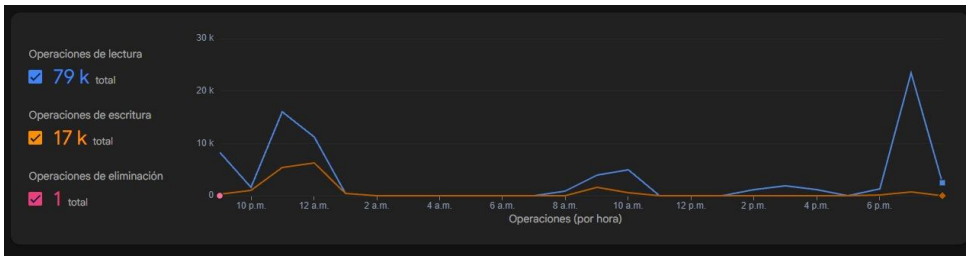
Consulta de /parking/sync-status: 23.728 eventos y cero pendientes.

Anexo B. Cuotas de Firestore



Panel de cuotas de Cloud Firestore, capturado el 28 de octubre de 2026.

Anexo C. Métricas de operaciones



Lecturas, escrituras y eliminaciones en el intervalo mostrado por Firestore.

Anexo D. Links del proyecto

- Repositorio de Github: https://github.com/CTorrejon1995/PTEC104-8571-S1-ESTACIONAMIENTO-INTELIGENTE
- Pagina web en Vercel: https://ptec-104-8571-s1-estacionamiento-in-eta.vercel.app/
    - **[admin]** christophertorrejon1995@gmail.com || clave: 123456
    - **[viewer]** mistersejaja@gmail.com || clave: 123456